CSC 2105::Data Structure

Searching - Sorting

Fahad Ahmed

Assistant Professor, Department of Computer Science

Faculty of Science & Information Technology

American International University-Bangladesh (AIUB)

fahad.ahmed@aiub.edu

SEARCHING ALGORITHM

- **Search**: locate an item in a list of data/information.
- Two algorithms/approaches will be examined –
  - Linear Search:
    - Searches sequentially for an element.
    - Starts from the first element.
  - Binary Search:
    - Searches an element by dividing the sorted elements into two groups.
    - Starts with the middle element.

LINEAR/SEQUENTIAL SEARCH

Concept

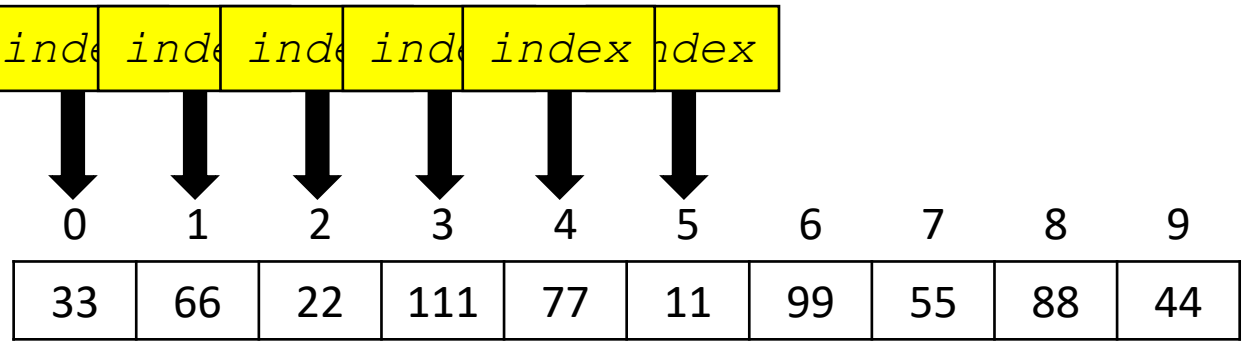
- Starts with the first element,
- Sequentially goes through a list of elements,
- Examines each element with the value to be found,
- Stops when –
  - It locates the value it is searching for (found).
  - No more elements to examine (not found).

```
int searchList(int list[], int numElems, int value){
    int index = 0; // Used as a subscript to search array
    int position = -1; // To record position of search value
    bool found = false; // Flag to indicate if value found

    while (index < numElems && !found){
        if (list[index] == value){ // If the value is found
            found = true; // Set the flag
            position = index; // Record the value's subscript
        }
        index++; // Go to the next element
    }

    return position; // Return the position, or -1
}
```

value	11
found	true
position	5



LINEAR/SEQUENTIAL SEARCH

- Benefits:
  - Easy algorithm to understand
  - Array can be in any order (value wise)
- Disadvantages:
  - Inefficient (slow): for array of N elements –
    - Examines N elements in worst case (last element was searched);
    - Examines 1 element in best case (1st element was searched);
    - Examines N/2 elements on average.
  - So for an array with N = 10000 element it will take N/2 = 5000 comparison on average to find an element

SORTING

- Sort: arrange values into an order:
  - Alphabetical
  - Ascending numeric
  - Descending numeric
- Three algorithms/approaches will be examined –
  - Bubble sort
  - Selection sort
  - Insertion sort

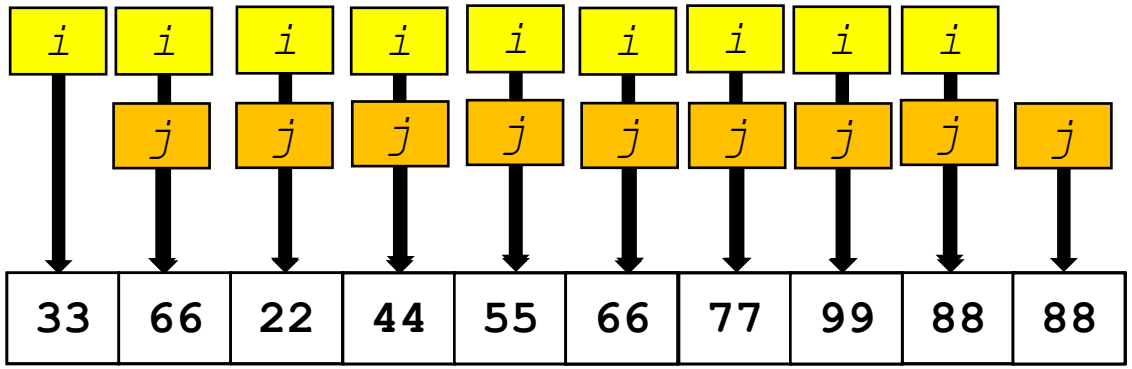
Bubble Sort

Concept

- The 1st index should contain the smallest value. Rest (2nd to the last) of the elements are compared with the 1st element.
- If any element is found to be smaller than the current 1st element then the elements are interchanged. At the end the 1st index contains the smallest value.
- Next the 2nd element is compared with rest (3rd to the last) of the elements and interchanged if found small. At the end the 2nd index contains the smallest value excluding the 1st element.
- Next the 3rd, 4th, …, (n-1)th elements are compared, interchanged, and smallest value is placed in appropriate index in the same manner as above.

```
/* sorting element in ascending order */
int i, j, tempVal, List[100];
/* find the smallest value for index i */
for (i=0; i<n-1; ++i)
/* compare with rest of the element in index j*/
    for (j=i+1; j<n; ++j)
        if(List[i] > List[j]){
/* interchange if value at index j is small  */
                tempVal = List[i];
                List[i] = List[j];
                List[j] = tempVal;
        }
```

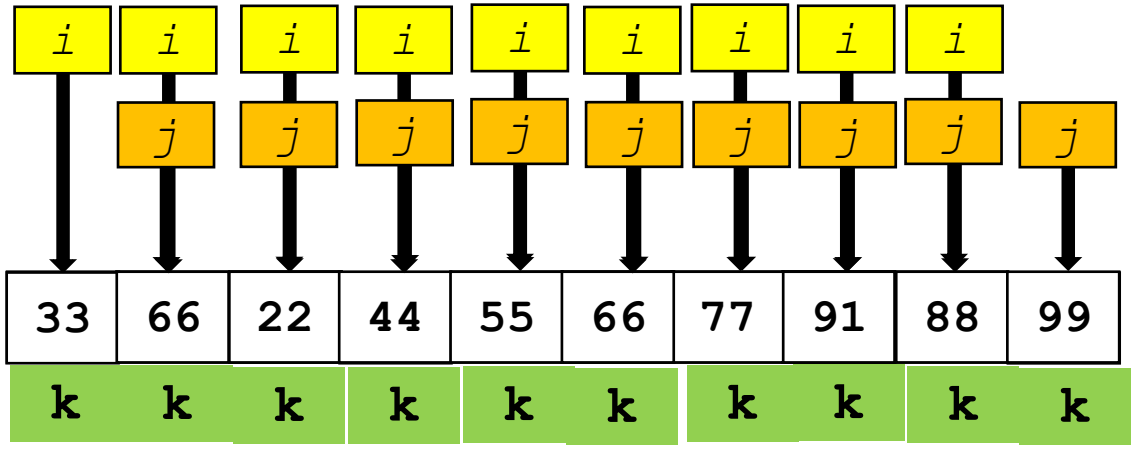
- **Benefit:** Easy to understand and implement
- **Disadvantage:** Inefficient – slow for large arrays



SELECTION SORT

Concept

- Locate smallest element in array. Interchange it with element in position 0.
- Locate next smallest element in array. Interchange it with element in position 1.
- Continue until all elements are arranged in order




```
/* sorting element in ascending order */
int i, j, k, tempVal, List[100];
/* find the smallest value for index i */
for ( i=0; i<n-1; ++i ){
    k = i; // consider value in k as smallest
/* compare with rest of the element in index j*/
    for ( j=i+1; j<n; ++j )
/* change k to j if value at index j is small  */
        if( List[k] > List[j] ) k = j;
/* interchange if value at index k is small  */
    tempVal = List[i];
    List[i] = List[k];
    List[k] = tempVal;
}
```

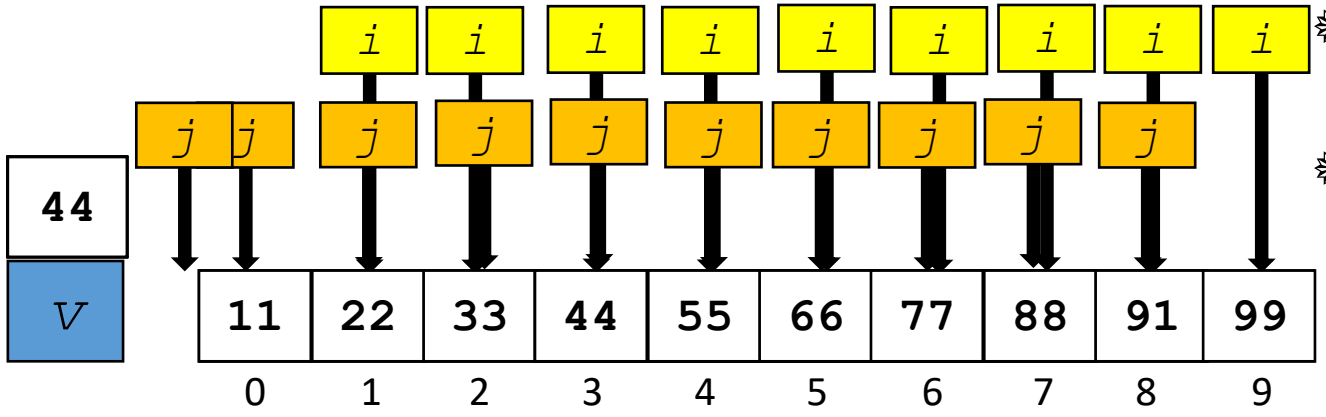
- **Benefit:** More efficient than Bubble Sort, since fewer exchanges
- **Disadvantage:** May not be as easy as Bubble Sort to understand

Insertion Sort

Concept

- Start with the 2nd element.
- Compare backwards with all previous (current index down to 0) elements.
- While the previous elements are large move the element one index forward.
- Stop and store the 2nd element where the previous element is not large.
- Continue with rest of the element forward as above.

```
/* sorting element in ascending order */
int i, j, v, List[100];
/* start with the 2nd element */
for ( i=1; i<n; ++i ){
    v = List[i]; // store the current value
    j = i - 1; // go for the previous element
/* compare with the previous elements in index j */
    while( j >= 0 && List[j] > v ){
    /* move forward if value at index j is large  */
        List[j+1] = List[j];
        j--; // go for the next previous element
    }
     List[j+1] = v; // store current value
}
```



- **Benefit:** May be efficient than Bubble Sort, but less than selection sort.
- **Disadvantage:** May not be as easy as other Sorting algorithm to understand

Binary Search

Concept

- Requires array elements to be sorted.
- Divides the array elements into three sections:
  - The middle element.
  - Elements on the left side of the middle element.
  - Elements on the right side of the middle element.
- If the middle element is the searched value, returns the position.
- Otherwise, any one side of the middle element is considered which might contain the searched element.
- Divide this side of the array as above and continue the previous steps until value is found or no more elements to be divided (not found).

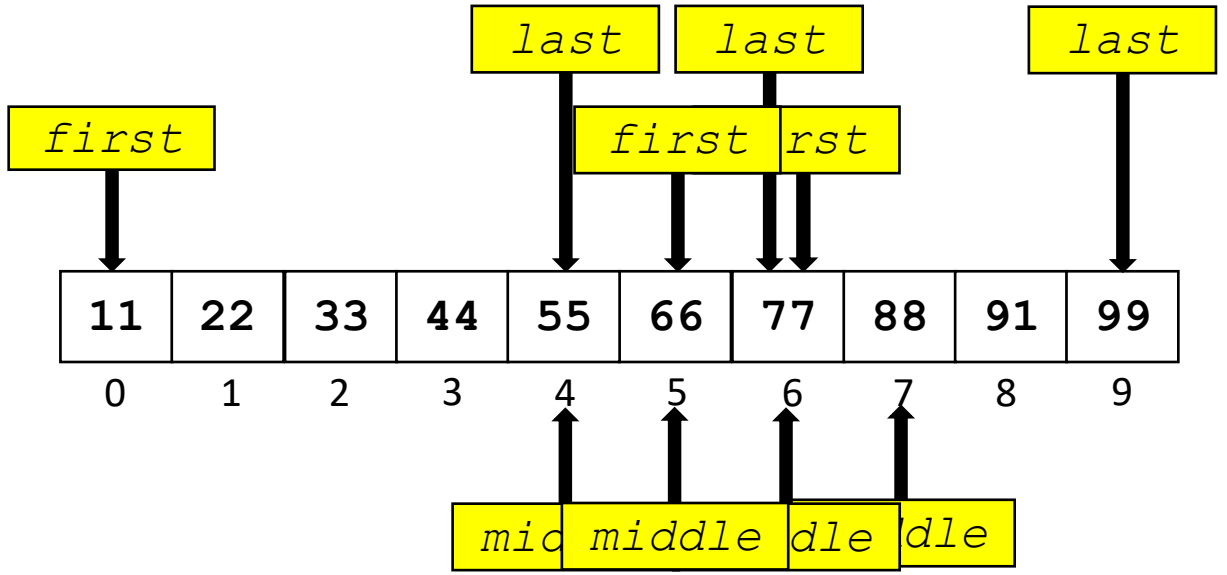
```
int binarySearch(int array[], int n, int value){
   int first = 0,     // First array element
       last = n - 1, // Last array element
       middle;       // Mid point of search
   while ( first <= last ){
      /* Calculate mid point */
      middle = (first + last) / 2;
      /* If value is found at mid */
      if (array[middle] == value)   return middle;
      else if (array[middle] > value)
         last = middle - 1; // If value is in lower half
      else
         first = middle + 1; // If value is in upper half
   }
   return -1;
}
```

BINARY SEARCH

value	60

first	5
last	4
middle	6

not found



- **Benefits**: Much more efficient than linear search. For array of N elements, performs at most $\log_2 N$ comparisons.
- **Disadvantages**: Requires that array elements be sorted